During the operation of the equipment, for the possibility of configuring and maintaining RAID arrays of Megaraid, LSI, Avago, Broadcom controllers in the AstraLinux OS environment versions 1.6, 1.7, 1.8 (https://ru.wikipedia.org/wiki/Astra_Linux), the task arose to deploy the LSA LSI Storage Authority Software. Software unification was not required

---

In my case, the account of the first user was used when installing the OS, which is equipped with sudo by default (if your user is not the first in the system, then resolve this issue with separate measures). Two implementation methods were chosen, which were performed in parallel.:

1.Search for compatible deb packages and install them on the above OS versions;

2.Search for package sources on the Broadcom website, build and install them, again for each OS separately (if it is impossible to build in AstraLinux, do it in the appropriate Debian version).

Considering the batch compatibility of AstraLinux versions 1.6/1.7/1.8 = Debian 9/10/11, taking into account the requirements for glibc no less than for the outdated version of the OS, it was possible to reach a version of the WebGUIRelease_Linux_8.004.010.000 software that meets the requirements of the task in terms of its characteristics.

However, an error occurred while installing the packages with a message about unsupported compression

---

In the course of clarifying the reasons for the failure to unpack the files, a solution was found that explained the nature of what was happening and the solutions, without the need for sysdig/ptrace debugging of what was happening. (I thank the authors of the posts for the arguments left on the Internet "https://unix.stackexchange.com/questions/669004/zst-compression-not-supported-by-apt-dpkg ", it is very clear and does not require a language translation).

```
If you are running Debian < 12 and need to install a .deb package that uses zstd, you can repack it:
Extract files from the archive
    ar x some-package.deb
Uncompress zstd files an re-compress them using xz
    zstd -d < control.tar.zst | xz > control.tar.xz
    zstd -d < data.tar.zst | xz > data.tar.xz
Re-create the Debian package in /tmp/
    ar -m -c -a sdsd /tmp/some-package.deb debian-binary control.tar.xz data.tar.xz
Clean up
    rm debian-binary control.tar.xz data.tar.xz control.tar.zst data.tar.zst
You should now be able to install the newly generated package:
    apt-get install /tmp/some-package.deb
```

Go to the directory of the unpacked archive WebGUIRelease_Linux_8.004.010.000.zip (find the specified release by going through the set of them at the link https://www.broadcom.com/site-search?page=7&per_page=10&q=lsa) and follow the steps outlined above by using the extended repository in apt.:

```
sudo apt update

sudo apt install zstd

ar x LSA_lib_utils-1.16-1_amd64.deb

zstd -d < control.tar.zst | xz > control.tar.xz

zstd -d < data.tar.zst | xz > data.tar.xz

ar -m -c -a sdsd /tmp/LSA_lib_utils-1.16-1_amd64.deb debian-binary control.tar.xz data.tar.xz

rm debian-binary control.tar.xz data.tar.xz control.tar.zst data.tar.zst

ar x LSA_lib_utils2-9.00-1_amd64.deb

zstd -d < control.tar.zst | xz > control.tar.xz

zstd -d < data.tar.zst | xz > data.tar.xz

ar -m -c -a sdsd /tmp/LSA_lib_utils2-9.00-1_amd64.deb debian-binary control.tar.xz data.tar.xz

rm debian-binary control.tar.xz data.tar.xz control.tar.zst data.tar.zst

Let's make all files executable (or make a command for files selectively):
```
chmod +x .

We will install the already repackaged software (for more information about the launch keys, see the file LSA_Linux_64_readme.txt):

---

sudo ./install_deb.sh -s (when asked about agreeing to the license, answer Y and press Enter)

```
Let's look at the status of the installed service:
      sudo systemctl status LsiSASH.service
      It works (*otherwise, you need to understand the reasons for what is happening)

Let's log in to management via the browser at:
     localhost:2463 ( or http://localhost:2463 ) or 127.0.0.1:2463 (or http://127.0.0.1:2463 ),
```
or use the address or alieses assigned to your network adapters.

The above actions can be performed on a variety of DEB-like operating systems, there are no restrictions here, be it Debian, Ubuntu and others. And here's the second surprise — when logging in, the user's username and password are requested.

In our case, this user must be root (not to be confused with the sudo capabilities of the user), moreover, verification via LDAP is used. The root user is not activated on our systems for security reasons.

The task was to start and manage the RAID. It was decided to configure the security of these connections separately upon completion of testing the mechanisms, by installing and configuring LDAP, configuring user rights, restrictions via iptables, or configuring the configuration file of this service, or even placing it in a sandbox (however, this is another story, no less intense than this one and requires separate consideration).

```
Stop the service:
```

sudo systemctl stop LsiSASH.service (please note it stops quickly, but starts within 15-20 seconds)

```
Disable autism:
    sudo nano /opt/lsi/LSIStorageAuthority/conf/LSA.conf
       find the line
            # bypass authentication (use with caution)
            bypass_authentication = 0
```

and replace with # bypass authentication (use with caution) bypass_authentication = 1
Press ctrl+o and enter (save the config)

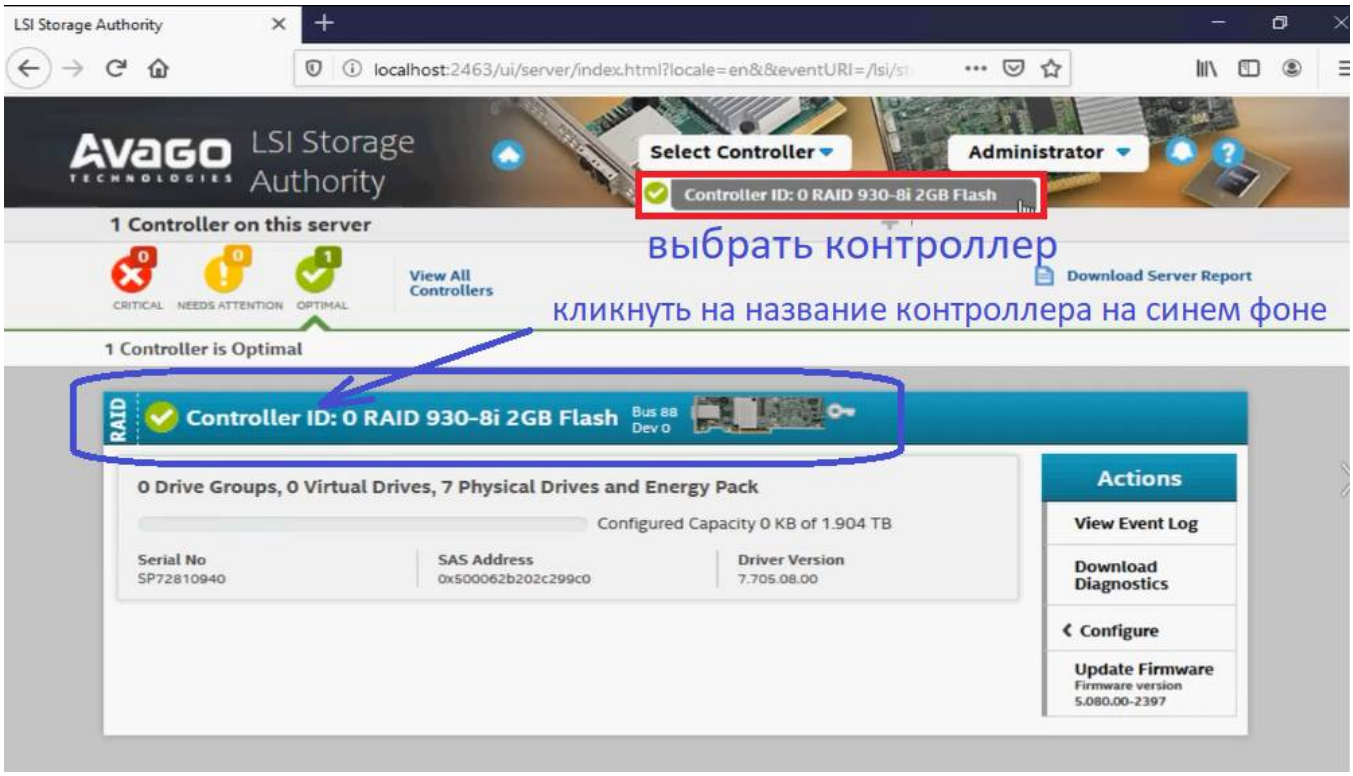
```
Let's start the service:

    sudo systemctl start LsiSASH.service
```

Let's log in to management via the browser at:

```
localhost:2463 ( or http://localhost:2463 ) or 127.0.0.1:2463 (or http://127.0.0.1:2463 ),
either use the address or alieses assigned to your network adapters.
```

In certain cases, you need to select a RAID adapter to work with, for example, if there are several of them. In the design, in the middle of the screen, you can see the name of the controller on a blue background, which in a normal environment seems to be the usual title of the built—in window, but here it works like a huge rectangular button with the name of the controller - feel free to click on the names and there will be page transitions (see the file "LSA LSI Storage Authority Software.png", forgive me I am the creators of this interface).



It would seem that what else is needed has finally worked out

During operation, when our service was ignored to manage the resources of a highly loaded I/O controller, situations with 100% of its load on processors/ cores often began to arise.

```
Stop the service:

    sudo systemctl stop LsiSASH.service

Deactivating the autorun of the service:

    sudo systemctl disable LsiSASH.service
```

The nature of what is happening has yet to be studied. It was found that the impact of 100% CPU utilization of our service does not occur immediately, but after a certain long period of time. It turns out that it is possible to view the status, replace the disk, rebuild the RAID, i.e. transfer the necessary commands to the controller and disconnect after the right time, but the service cannot be left running on an ongoing basis, so as not to load the processor at 100%.

Let's create a snap-in to automatically start our service, manage arrays, and stop the service on servers from the operator's workplace via a pseudographic GUI interface in bash

To do this, install the necessary packages.:

```
sudo apt install dialog sshpass firefox

where:
dialog is a utility for building console interfaces;
sshpass is a utility for running ssh using password authentication mode in non–interactive mode.;
firefox is a web browser.
```

Create a bash script "lsacontrol.sh ", in which:

```
 We check whether the necessary auxiliary utilities are installed.

 We provide a list of servers with descriptions (change the context to your IP address and comments)

 We specify the password and username (it's not safe, rewrite this place)

 Let's create a function for checking the status of the LSA service

 We are preparing the menu for dialog (# Server selection dialog - play with the parameters "15 60
6" to make everything fit in)

 Initiating the server selection dialog

 Checking the server selection

 Next, we check the availability of the server
```

When it is available, we run the LSA service on it.

We are waiting for the launch of the LSA service (it needs to be made a separate function, I wrote earlier about the long launch of the service)

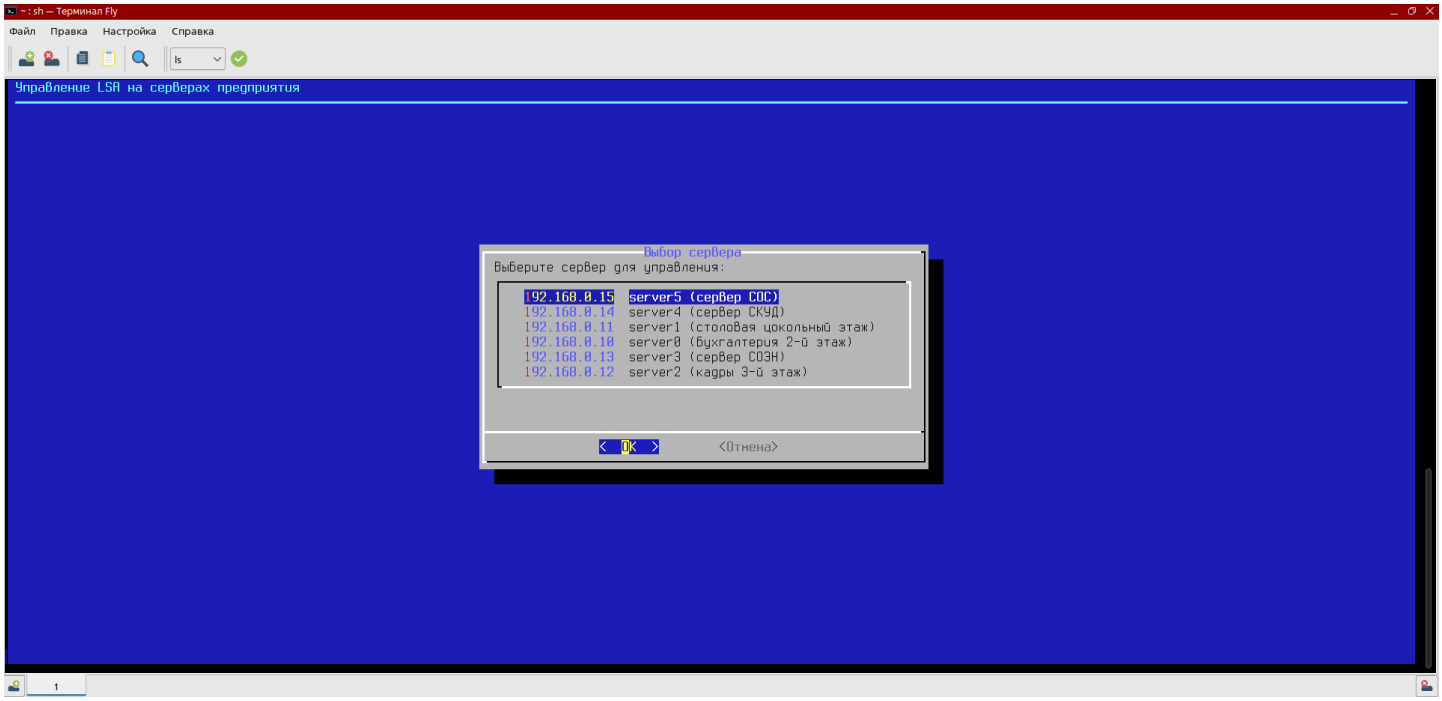
Upon successful launch of the LSA service on the selected server, we launch Firefox with the IP address of the selected server and port :2464

Tracking the pid of a previously launched Firefox window

After completing work with the controllers/arrays and closing the Firefox window, we send a command to the server to stop the service.

Start management via "lsacontrol.sh ":

```
admin@astra1$  sh ./lsacontrol.sh
```



**You should pay special attention to port 9000. It is used for internal exchange of LsiSASH service. We had a high-performance MinIO distributed object data warehouse (S3 compatibility) running on our servers on this port. The ss utility helped, which shows not only the ports, but also the process that occupied them. Reconfigured the MinIO to a different port. Other processes may also use it, take action. Read the file LSA_Linux_64_readme.txt - it specifies the installation modes and the ability to change the default ports (section "Installation Instructions").

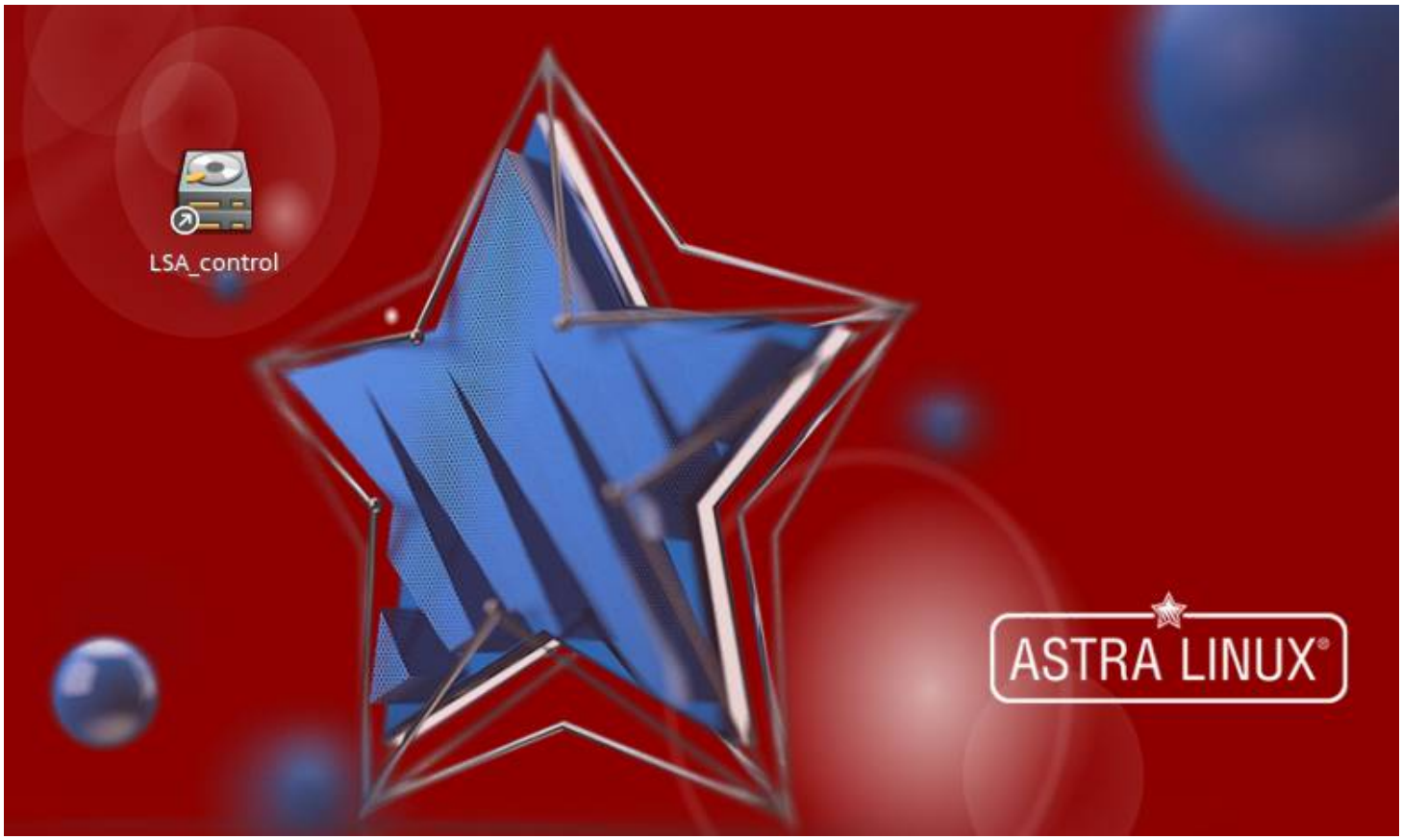
#Zoom to display the dialog box. Sometimes the dialog menu is duplicated and displayed below. To display correctly, you need to use a mouse-type manipulator to resize the window or scroll to change the font size in the window, in both cases individually increase or decrease them.

In any case, before using it, it is necessary to finalize all of the above according to the safety requirements of the environment where it will function.

Create a shortcut to easily launch our project.

```
nano /home/admin/Desktop/LSA.desktop

[Desktop Entry]
Name=LSA_control
Comment=cortrol raids
GenericName=Megaraid, Lsi, Avago, Broadcom, LSA, RAID
Keywords=raids
Exec=sh /home/admin/LSA/lsacontrol.sh
Terminal=true
Type=Application
Icon=/usr/share/icons/fly-astra/256x256/devices/raid.png
Path=
Categories=
NoDisplay=false
```



For all emergency situations and malfunctions, it is necessary to return to the initial position of the "deactivated" and "stopped" service on the selected server:

```
sudo systemctl disable LsiSASH.service
sudo systemctl stop LsiSASH.service
```

Next, run the control via ”lsacontrol.sh ” (it is necessary to have sudo rights for individual commands):

```
admin@astra1$ sh ./lsacontrol.sh
```

The need to assemble LSA packages from source, as the second method of solving the problem, will now be only in case of unsolvable malfunctions during operation. By coincidence, this solution, as a unified software, was tested and applied on the AstraLinux OS on versions 1.6, 1.7, 1.8 with a variety of cores.

To deploy LSA in the AstraLinux OS, a file with installation instructions is posted ”howtoinst-en.txt ” and prepared repackaged files.

This solution can be applied to newer versions of LSA (https://www.broadcom.com/site-search?page=7&per_page=10&q=lsa). Please pay attention to the need to use a VPN in a situation with difficulty displaying a resource.

Successful integration and operation